# Section 1 - Computer Arithmetic & Numerical Accuracy

**M1 Mandelbrot Trajectory Divergence**

To be able to see the impact of the numerical accuracy a trajectory divergence is performed on the Mandelbrot set using the complex128 and complex64 data types. The trajectory divergence test was done in the Seahorse Valley, a region where the complex constant C is initialized as $R = [-0.753;\ -0.749]$ and $I = [0.099;\ 0.103]$. The test was performed with a Mandelbrot set with the resolution being 1024x1024, with the maximum number of iterations being 1000, and divergence threshold $\tau = 0.01$. The code can be found in mandelbrot_trajectory_divergence.py.

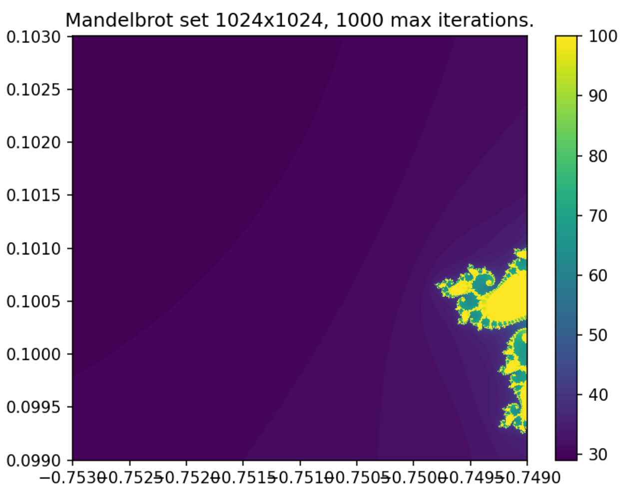
*What is the fraction of pixels that diverge before max iter?*

I find that the fraction of pixels diverging before the maximum number of iterations is $0.9999981$, so nearly all pixels diverge before reaching the maximum number of iterations. In total only two pixels reaches the maximum number of iterations.

If we instead set the maximum number of iterations to 100, the fraction becomes $0.9846$.

*Where do trajectories diverge early? Compare visually to the escape-count map.*

By looking at just Figure 1, it seems that the trajectory diverges early in exterior areas of the Mandelbrot set, but also in the some of the interior areas. However, looking at Figure 2, it can be clearly seen that this is not the case, as the maximum iteration before divergence is consistently reached across the interior of the set. This means that in the exterior of the set and the border areas the numerical accuracy quickly deteriorates.



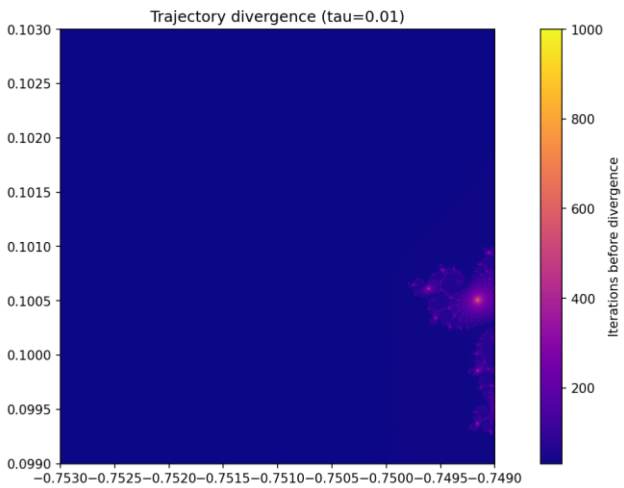




*Figure 1, (Left) The Seahorse Valley of the Mandelbrot set. (Right) The trajectory divergence map of the Seahorse Valley.*

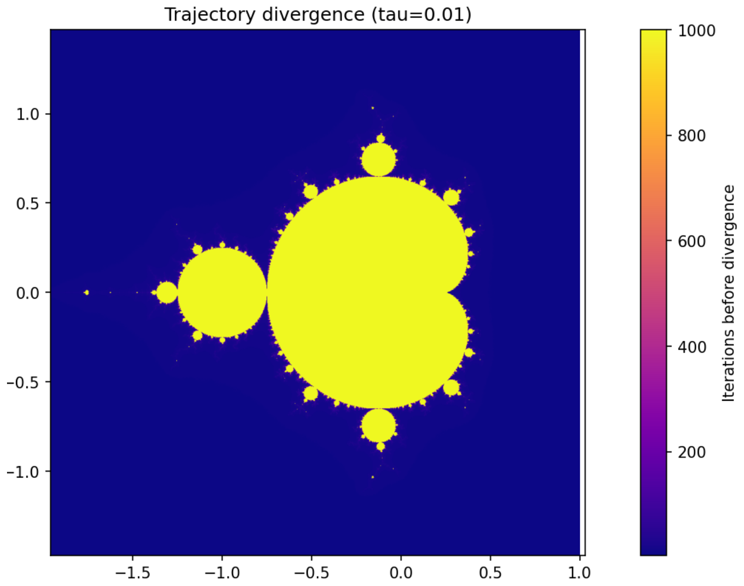



*Figure 2, Divergence map for the entire Mandelbrot Set.*

*Does early divergence correlate with high escape iteration counts?*

No, it does not. It is the mostly the opposite, which means high escape iteration count correlate with late divergence and vice versa.

**M2 Mandelbrot Sensitivity map**

The sensitivity map is computed by calculating $\kappa$, which approximates the relative change in input to the relative change in output.

$$\kappa(c) = \frac{|\Delta n|}{\epsilon_{32} \cdot n(c)}$$

$$\Delta n = n(c + \delta) - n(c)$$

$$\delta = \epsilon_{32} \cdot |c|$$

$c$ is a complex constant.

$\epsilon_{32}$ is floating-point accuracy, defined as the difference between 1.0 and the next float of the same size that can be represented.

$\delta$ is a perturbation to the complex constant.

$n(c)$ is the number of iterations it takes before a pixel escapes.

$\Delta n$ is the change in the number of iterations before a pixel escape, when adding the pertubation $\delta$ to the complex constant $c$ and subtracting that from the$n(c)$.

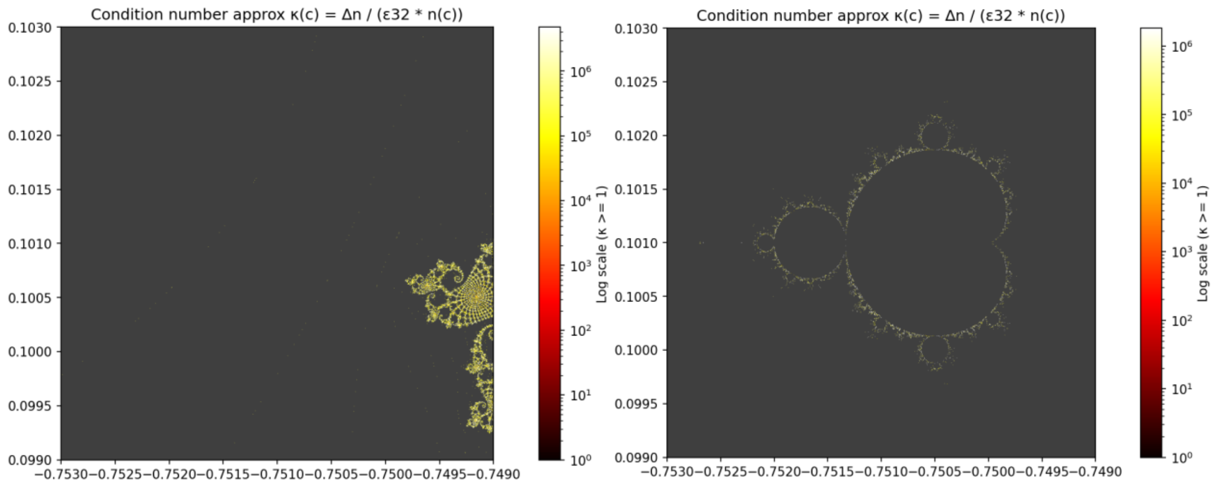



*Figure 3, Sensitivity map for: (Left) Seahorse Valley, (Right) The entire Mandelbrot set.*

*Where is κ largest? Does it match the boundary in M1?*

When looking at Figure 3 left, $\kappa$ seems to be largest near the boundaries and in the interior. However, when looking at Figure 3 right, it can be clearly seen that $\kappa$ is largest in the boundary areas and stable elsewhere. This matches with M1, except for the exterior. This may be because the complex number in the exterior quickly grows beyond the value it would have escaped with in the normal Mandelbrot Set.

*What is κ for interior pixels (n = max iter)?*

Again, when looking at Figure 3 left, it is difficult to determine. However, on Figure 3 right, $\kappa$ is clearly small for the interior pixels. This means that the problem is well-conditioned only in the non-boundary. Whereas the problem is ill-conditioned in the boundary areas, which means that small input perturbations $\delta$ propagate and become large output errors $\kappa$.

# Section 2 - Testing & Documentation

### M1 Test Suite

I created a test suite consisting of 6 unit tests. All of the tests passed after I fixed a few undiscovered bugs that were visually undetectable. The coverage of the unit tests was quite low at only 11%. The reason for this low coverage is because my project folder contains both test files as well as exercise files, such as the Monte Carlo pi estimation, as seen on Figure 4. These files do not need testing, as they are not part of the Mandelbrot project, making the coverage quite meaningless in this case.

```
==================== test session starts ====================
platform win32 -- Python 3.11.14, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\srisb\.local\share\mamba\envs\nsc2026\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\srisb\OneDrive\Dokumenter\Alt(Rog)\Universitet\8.Semester\Numerical Scientific Computing\nsc-course
plugins: cov-7.1.0
collected 8 items

test_mandelbrot.py::test_numba_mandelbrot_point[0j-100] PASSED                                   [ 12%]
test_mandelbrot.py::test_numba_mandelbrot_point[(-1+1j)-2] PASSED                                [ 25%]
test_mandelbrot.py::test_numba_mandelbrot_point[(-1.5+0.01j)-13] PASSED                          [ 37%]
test_mandelbrot.py::test_mandelbrot_mp_chunk_output PASSED                                       [ 50%]
test_mandelbrot.py::test_mandelbrot_mp_chunk_output_length PASSED                                [ 62%]
test_mandelbrot.py::test_mandelbrot_vec_correctness PASSED                                       [ 75%]
test_mandelbrot.py::test_mandelbrot_numba_correctness PASSED                                     [ 87%]
test_mandelbrot.py::test_numerical_differences PASSED                                            [100%]

==================== tests coverage ====================
____________________ coverage: platform win32, python 3.11.14-final-0 ____________________

Name                                    Stmts   Miss  Cover
-----------------------------------------------------------
dask_test.py                                7      5    29%
julia.py                                   39     39     0%
line_profile.py                            16     16     0%
mandelbrot_dask.py                         74     74     0%
mandelbrot_dask_distributed.py            104    104     0%
mandelbrot_dask_distributed_sweep.py      106    106     0%
mandelbrot_dask_sweep.py                  104    104     0%
mandelbrot_mp.py                           90     77    14%
mandelbrot_mp_chunked.py                   97     97     0%
mandelbrot_naive.py                        47     20    57%
mandelbrot_numba.py                        50     40    20%
mandelbrot_numba_parallel.py               49     49     0%
mandelbrot_sensitivity_map.py              36     36     0%
mandelbrot_time_scaling.py                 18     18     0%
mandelbrot_trajectory_divergence.py        37     37     0%
mandelbrot_vectorized.py                   41     21    49%
memory_layout_performance.py               16     16     0%
memory_type_performance.py                 21     21     0%
monte_carlo_pi_dask.py                     33     33     0%
monte_carlo_pi_est.py                      52     52     0%
numba_benchmark.py                         22     22     0%
profiling.py                               21     21     0%
test_install.py                            13      0   100%
test_mandelbrot.py                         46      1    98%
-----------------------------------------------------------
TOTAL                                    1139   1009    11%
==================== 8 passed in 129.68s (0:02:09) ====================
```

*Figure 4 pytest test coverage.*

### M2 Docstrings and Type Hints

Docstrings on the NumPy style format was added to mandelbrot_naive.py along with type hints for both parameters and return variables. A slight problem with type hinting NumPy arrays is that NumPy does not specify how to type hint the shape an array. It is possible to type hint the array type, but as this can also change, I have just used the basic np.ndarray type. All ruff checks were passed as shown in Figure 5.

```
(nsc2026) C:\Users\srisb\OneDrive\Dokumenter\Alt(Rog)\Universitet\8.Semester\Numerical Scientific Computing\nsc-course>ruff check mandelbrot_naive.py
All checks passed!
```

*Figure 5 Verification that all ruff checks were passed in mandelbrot_naive.py.*

# Section 3 – PyOpenCL

123